一、完全平方

1. 问题分析

给定一个整数 n，需要找到最少数量的完全平方数（如 $1, 4, 9, 16, \cdots$），使得它们的和恰好等于 n。

（1）输入：一个非负整数 n（通常 n ⩾ 0）。

（2）输出：最少数量。

（3）示例：n = 12 → 输出 3（因为 12 = 4 + 4 + 4 或 9 + 1 + 1 + 1，最少数量为 3）。

（4）约束：典型的动态规划问题，n 的大小决定了算法效率，本解法适用于 n 在万级别以内。

2. 算法思想

采用**动态规划**（DP）自底向上求解。

2.1 状态定义

dp[i] 表示组成整数 i 所需的最少完全平方数个数。

2.2 初始化

dp[0] = 0（0 不需要任何数）；其余 dp[i] 初始化为一个较大的值（如 INT_MAX-1）。

2.3 状态转移方程

对于每个 i（1 ⩽ i ⩽ n），考虑所有可能的完全平方数 j^2 ⩽ i，尝试用 dp[i - j^2] + 1 更新 dp[i]，即：

$$dp[i] = \min_{j \geq 1,\ j^2 \leq i} \{dp[i - j^2] + 1\}$$

2.4 动态规划性质分析

动态规划能够正确求解该问题的前提是问题具备以下两个关键性质：

（1）最优子结构

性质：问题的最优解包含其子问题的最优解。
证明：设 dp[i] 是组成 i 的最少平方数个数，并且假设在最优解中选取的第一个平方数为 k^2（k^2 ⩽ i）。那么剩余部分 i - k^2 的平方数组成方案一定是组成 i - k^2 的最优解（即 dp[i - k^2]）。
反证：如果剩余部分存在更优方案（数量更少），则将该方案与 k^2 组合将得到比 dp[i] 更少的平方数个数，与 dp[i] 是最优矛盾。因此，问题满足最优子结构。

（2）重叠子问题

性质：在递归求解过程中，相同的子问题会被多次计算。
说明：例如计算 dp[12] 时，需要知道 dp[11]、dp[8]、dp[3] 等；而计算 dp[11] 时又需要 dp[10]、dp[7]、dp[2] 等。这些子问题被反复依赖，产生大量重复计算。
优势：动态规划通过自底向上填表（或带备忘录的自顶向下）将每个子问题的解只计算一次并存储，从而避免指数级递归开销。

由于同时满足最优子结构和重叠子问题，动态规划是求解该问题的有效方法。

2.5 算法流程

- 初始化 dp[0] = 0，其余 dp[i] 为一个大数。
- 外层循环 i = 1 到 n：
    - 内层循环 j = 1 到 sqrt(i)：
        - 更新 dp[i] = min(dp[i], dp[i - j*j] + 1)
- 返回 dp[n]。

3. 算法伪代码

```
函数 numSquares(n):
    创建数组 dp[0..n]
    初始化 dp[0] = 0
    对于 i 从 1 到 n:
        dp[i] = INF

    对于 i 从 1 到 n:
        j = 1
        当 j*j ≤ i 时:
            dp[i] = min(dp[i], dp[i - j*j] + 1)
            j = j + 1

    返回 dp[n]
```

4. 复杂度分析

（1）时间复杂度：外层循环遍历 1..n，内层循环对于每个 i 遍历所有满足 $j^2 \leqslant i$ 的 j，即大约 √i 次。总迭代次数为：

$$\sum_{i=1}^{n} \lfloor \sqrt{i} \rfloor = O(n\sqrt{n})$$

因此时间复杂度为 **O(n√n)**。

（2）空间复杂度：使用了一个长度为 n+1 的数组 dp，因此空间复杂度为 **O(n)**。

该解法对于中等规模的 n（如 $n \leqslant 10^4$）可高效运行。若 n 极大，可考虑数学方法（四平方定理）进一步优化。

二、最大子数组和

1. 问题分析

给定一个整数数组 nums，需要找到一个**连续子数组**（至少包含一个元素），使得该子数组的元素和最大，并返回这个最大和。

（1）输入：整数数组 nums，长度 ⩾ 1。

（2）输出：最大子数组和。

（3）示例：nums = [-2,1,-3,4,-1,2,1,-5,4] → 输出 6（对应子数组 [4,-1,2,1]）。

（4）特点：数组中可能包含负数，因此子数组不能随意扩展，需要动态决策。

2. 算法思想

采用**动态规划（Kadane 算法）**，通过一次扫描即可求解。

2.1 状态定义

dp[i] 表示**以** nums[i] **结尾**的连续子数组的最大和。
注意：dp[i] 必须包含 nums[i] 作为最后一个元素。

2.2 初始化

dp[0] = nums[0]（只有一个元素时，最大和就是它本身）。

2.3 状态转移方程

对于 i ⩾ 1，以 i 结尾的子数组有两种可能：

1. 只包含 nums[i] 自身；

2. 将 nums[i] 接在以 i-1 结尾的最大子数组后面。

因此：

$$dp[i] = \max(nums[i], dp[i-1] + nums[i])$$

也可以写作：

$$dp[i] = nums[i] + \max(0, dp[i-1])$$

（如果 dp[i-1] < 0，则重新开始；否则延续之前的结果。）

最终答案为：

$$\max_{0 \le i < n} dp[i]$$

2.4 动态规划性质分析

（1）最优子结构

性质：问题的最优解包含子问题的最优解。
证明：设 dp[i] 是以 i 结尾的最大子数组和。若 dp[i-1] > 0，那么 dp[i] = dp[i-1] + nums[i] 必然是最优的，因为任何不以 i-1 结尾的更优子数组都无法与 nums[i] 连接（会破坏连续性）。若 dp[i-1] ⩽ 0，则单独以 nums[i] 开头更优。因此，dp[i] 的构造仅依赖于 dp[i-1] 和 nums[i]，满足最优子结构。

（2）重叠子问题

性质：在递归求解过程中，子问题 dp[0], dp[1], ..., dp[i-1] 会被多次依赖。
说明：若采用分治或递归，计算 dp[i] 会重复计算 dp[i-1]，而 dp[i-1] 又依赖于 dp[i-2]，形成链式重复。动态规划通过自底向上迭代，每个 dp[i] 只计算一次，避免了指数级开销。

由于满足最优子结构和重叠子问题，动态规划（Kadane 算法）是求解该问题的标准方法，且能达到 O(n) 时间。

2.5 算法流程

- 初始化 sum = INT_MIN（记录全局最大和），b = INT_MIN（记录以当前元素结尾的最大和）。
- 遍历数组 nums：
    - 如果 b > 0，则 b += nums[i]（延续之前的子数组）；
    - 否则 b = nums[i]（重新开始子数组）。

    - 更新 sum = max(sum, b)。
- 返回 sum。

3. 算法伪代码

```
函数 maxSubArray(nums):
    sum = -∞
    b = -∞
    对于 i 从 0 到 nums.length-1:
        如果 b > 0:
            b = b + nums[i]
        否则:
            b = nums[i]
        如果 b > sum:
            sum = b
    返回 sum
```

4. 复杂度分析

（1）时间复杂度：仅需一次遍历数组，每个元素执行常数时间操作，因此时间复杂度为 **O(n)**，其中 n 为数组长度。

（2）空间复杂度：只使用了两个额外变量 sum 和 b，与输入规模无关，因此空间复杂度为 **O(1)**。

三、零钱兑换

1. 问题分析

给定一个整数数组 nums（表示不同面额的硬币，每种硬币无限可用）和一个整数 n（总金额），需要找到凑成 n 所需的**最少硬币个数**。如果无法凑成，返回 -1。

（1）示例：nums = [1, 2, 5]，n = 11 → 输出 3（11 = 5 + 5 + 1）。

（2）示例：nums = [2]，n = 3 → 输出 -1（无法凑成）。

（3）特点：每种硬币无限，属于**完全背包**问题；目标是求最小值，不是组合数。

2. 算法思想

采用**动态规划（完全背包）**，使用一维数组优化空间。

2.1 状态定义

dp[j] 表示凑成金额 j 所需的最少硬币数量。

2.2 初始化

1. dp[0] = 0（凑 0 元不需要硬币）。

2. 其他 dp[j] 初始化为 INT_MAX - 1（一个大值，用于表示不可达，且避免后续加 1 时溢出）。

2.3 状态转移方程

对于每个硬币面额 nums[i]，遍历金额 j 从 nums[i] 到 n（正序）：

$$dp[j] = \min(dp[j], dp[j - nums[i]] + 1)$$

解释：要么不使用当前硬币（保持 dp[j]），要么使用一枚当前硬币，然后加上凑成 j - nums[i] 的最少硬币数。

注：为什么外层循环硬币、内层循环金额（正序）？

1. 完全背包中每种硬币无限，正序遍历金额可以重复使用同一枚硬币（与 0-1 背包的倒序遍历相反）。

2. 最终 dp[n] 即为答案，若 dp[n] 仍等于 INT_MAX - 1（即不可达），则返回 -1。

2.4 动态规划性质分析

（1）最优子结构

性质：问题的最优解包含子问题的最优解。
证明：设凑成金额 j 的最优方案中使用了某种面额为 coin 的硬币。那么去掉这枚硬币后，剩余金额 j - coin 的凑法必然是凑成 j - coin 的最优方案（否则可以替换得到更优的 j 方案）。因此，状态转移方程基于子问题 dp[j - coin] 的最优解，满足最优子结构。

（2）重叠子问题

性质：在递归求解过程中，同一子问题会被多次计算。
说明：例如凑金额 10 时，需要知道金额 9、8、5 等；而凑金额 9 又需要金额 8、7、4 等。大量子问题重复出现。动态规划通过自底向上填表，每个 dp[j] 只计算一次，避免指数级递归开销。

由于同时满足最优子结构和重叠子问题，动态规划是求解该问题的有效方法。

2.5 算法流程

- 初始化 dp 数组长度为 n+1，所有值设为 INT_MAX - 1，dp[0] = 0。
- 外层遍历每个硬币 nums[i]:
    - 内层 j 从 nums[i] 到 n（正序）:
        - dp[j] = min(dp[j], dp[j - nums[i]] + 1)
- 若 dp[n] == INT_MAX - 1 返回 -1，否则返回 dp[n]。

3. 算法伪代码

```
函数 coinChange(nums, n):
    创建数组 dp[0..n]
    初始化 dp[0] = 0
    对于 j 从 1 到 n:
        dp[j] = INT_MAX - 1

    对于 i 从 0 到 nums.length - 1:
        对于 j 从 nums[i] 到 n:
            dp[j] = min(dp[j], dp[j - nums[i]] + 1)

    如果 dp[n] == INT_MAX - 1:
        返回 -1
    否则:
        返回 dp[n]
```

4. 复杂度分析

（1）时间复杂度: 外层循环遍历 m 种硬币（m = nums.length），内层循环遍历金额 O(n)，总时间复杂度为 **O(m × n)**。

（2）空间复杂度：使用了一个长度为 n+1 的一维数组，因此空间复杂度为 **O(n)**。

该解法是求解完全背包最小值问题的标准写法，适用于 n 在万级别以内、硬币种类适中的场景。若 n 极大，可考虑 BFS 或数学优化方法。

四、最长递增子序列

整体问题分析

给定一个整数数组 nums，需要找到其中**最长严格递增子序列**的长度。子序列不要求连续，但顺序必须与原数组一致。

1. 示例：[10,9,2,5,3,7,101,18] → 输出 4（子序列 [2,3,7,101] 或 [2,5,7,101]）。

2. 特点：元素可删除任意位置，只需保持相对顺序。

（一）动态规划法（$O(n^2)$）

1. 问题分析

采用动态规划，直接定义状态：以每个元素结尾的最长递增子序列长度。通过遍历所有之前的元素进行转移。

2. 算法思想

2.1 状态定义

dp[i] 表示以 nums[i] 结尾的**最长严格递增子序列的长度**。

2.2 初始化

每个元素自身可以构成长度为 1 的子序列，故 dp[i] = 1 对所有 i。

2.3 状态转移

对于每个 i，考虑所有 j < i:

1. 若 nums[j] < nums[i]，则可以将 nums[i] 接在以 nums[j] 结尾的子序列后面，形成长度 dp[j] + 1。

2. 取所有可能中的最大值:

$$dp[i] = \max_{0 \le j < i,\ nums[j] < nums[i]} \{dp[j] + 1\}$$

3. 最终答案为 max(dp[0], dp[1], ..., dp[n-1])。

2.4 动态规划性质分析

1. 最优子结构：以 i 结尾的最优子序列，去掉最后一个元素后，必然是以某个 j < i 且 nums[j] < nums[i] 结尾的最优子序列。否则可以替换得到更长序列，满足最优子结构。

2. 重叠子问题：计算不同 i 时需要重复扫描之前的 j，存在大量重复比较。动态规划通过填表避免递归重复计算。

2.5 算法流程

获取数组长度 n，创建 dp 数组并全部初始化为 1。

- 外层循环 i = 0 到 n-1:
    - 内层循环 j = 0 到 i-1:
        - 如果 nums[j] < nums[i]，则 dp[i] = max(dp[i], dp[j] + 1)
- 遍历 dp 找出最大值 maxm，返回 maxm。

3. 算法伪代码

```
函数 lengthOfLIS(nums):
    n = nums.length
    dp = 数组长度为 n，所有元素初始化为 1
    for i = 0 到 n-1:
        for j = 0 到 i-1:
            if nums[j] < nums[i]:
                dp[i] = max(dp[i], dp[j] + 1)
    maxm = 0
    for i = 0 到 n-1:
        maxm = max(maxm, dp[i])
    return maxm
```

4. 复杂度分析

（1）时间复杂度：两层循环，总比较次数为 n(n-1)/2，因此为 **$O(n^2)$**。

（2）空间复杂度：使用了一个长度为 n 的 dp 数组，**O(n)**。

（二）贪心 + 二分查找法（O(n log n)）

1. 问题分析

为了优化到 O(n log n)，需要换一种思路：维护一个数组 tail，其中 tail[k] 表示长度

为 k+1 的递增子序列的**最小可能末尾元素**。通过二分查找确定当前元素应放置的位置。

2. 算法思想

2.1 贪心性质

1. 对于一个长度相同的递增子序列，末尾元素越小，越有利于后续元素扩展出更长的子序列。

2. 因此我们维护 tail 数组，始终保持每个长度的最小末尾值。

2.2 算法流程

- 初始化空数组 tail。
- 遍历每个 num：
    - 在 tail 中二分查找**第一个大于等于** num 的位置 pos（即左边界）。
    - 若 pos == tail.size()，说明 num 比所有已存在的末尾都大，可以扩展最长长度，将 num 追加到 tail 末尾。
    - 否则，将 tail[pos] 替换为 num（因为 num 更小，更优）。
- 最终 tail 的长度即为最长递增子序列的长度。

注：tail 数组本身并不一定是一个合法子序列，但其长度正确。

2.3 算法流程

- 获取数组长度 n，创建空 vector<int> tail。
- 对于每个 num in nums：
    - 设置 left = 0, right = tail.size()，pos = tail.size()。
    - 二分查找：while (left < right)，计算 mid，若 tail[mid] >= num，则 right = mid, pos = mid；否则 left = mid + 1。
    - 若 pos == tail.size()，则 tail.push_back(num)；
    - 否则 tail[pos] = num。
- 返回 tail.size()。

3. 算法伪代码

函数 lengthOfLIS(nums):

    tail = 空数组

    for 每个 num in nums:

```
        left = 0
        right = tail.length
        pos = tail.length
        while left < right:
            mid = (left + right) / 2
            if tail[mid] >= num:
                right = mid
                pos = mid
            else:
                left = mid + 1
        if pos == tail.length:
            tail.append(num)
        else:
            tail[pos] = num
    return tail.length
```

4. 复杂度分析

（1）时间复杂度：每个元素进行一次二分查找，二分查找复杂度为 O(log L)，其中 L 为当前 tail 长度（≤ n）。总复杂度为 **O(n log n)**。

（2）空间复杂度：使用了 tail 数组，最多存储 n 个元素，**O(n)**。